

REMO: Deterministic Scenario Record and Replay with Modification for ADS Debugging in CARLA

Matthew I Leach¹, Sanjeetha Pennada¹, and Donghwan Shin¹

¹ School of Computer Science, University of Sheffield, United Kingdom. Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

In simulation-based testing of autonomous driving systems (ADS), the ability to reproducibly recreate and modify driving scenarios is crucial for testing and debugging system performance. For example, when an ADS fails to navigate a complex urban driving scenario, researchers and engineers need to isolate the contributing factors by systematically modifying scenario elements, such as weather conditions, time of day, street lights, traffic signs, or the presence of other vehicles and pedestrians. However, existing simulation platforms like CARLA (Dosovitskiy et al., 2017) often lack the capability to deterministically replay scenarios with modified parameters while maintaining consistent non-player character (NPC) behaviour.

To address this limitation, we present REMO, a tool that enables deterministic replay and modification of simulation scenarios within CARLA. REMO provides the following key features:

- Deterministic Scenario Replay:** REMO allows users to record and replay any driving scenarios in CARLA while ensuring that all static and dynamic elements, including NPC behaviours, remain consistent across replays, unless intentionally modified.
- Scenario Modification:** Users can modify various scenario elements/parameters, including environmental conditions (e.g. weather, time of day) and the presence of specific actors (e.g. vehicles, pedestrians), without disrupting the deterministic nature of the replay.
- ADS-Agnostic Testing:** REMO supports scenario replay with or without an ego vehicle (i.e. the autonomous vehicle under test), enabling researchers to test different ADS models under identical NPC conditions for fair comparisons. By enabling deterministic replay and flexible scenario modification, REMO facilitates systematic debugging and evaluation of autonomous driving systems.

Statement of Need

Debugging failures in machine learning-enabled autonomous driving systems (ADS) is a complex task that requires the ability to reproducibly recreate and modify driving scenarios (Shin & Pennada, 2024). For example, Arcaini et al. (2022) proposed iterative removal of NPC vehicles to isolate the minimal set of NPCs responsible for triggering a failure, showing that simplified scenarios significantly aid engineers during debugging. However, they focused on NPC removal and did not address the broader need for deterministic scenario replay and modification of other scenario elements (e.g. weather, time of day, buildings). Furthermore, due to the inherent non-determinism of simulation platforms (Chance et al., 2022; Osikowicz et al., 2025), simply re-running a scenario without explicitly recording and replaying it can lead to different outcomes, making it difficult to isolate failure causes. Although CARLA supports record and replay functionality, it lacks the ability to modify scenarios while maintaining deterministic NPC behaviour.

REMO is designed to fill this gap by providing a tool that enables deterministic replay of recorded scenarios with the ability to modify various scenario parameters without affecting NPC behaviour. It allows users to systematically explore how different factors contribute

to ADS failures by enabling controlled modifications of the simulation environment. It also supports ADS-agnostic testing by allowing scenario replay without an ego vehicle, facilitating fair comparisons between different ADS models under identical conditions.

Overview of the Tool

Figure 1 illustrates the overall architecture of REMO. (More to be added later; please mention key components that enable the features described above. This can also be added to the repository README later.)

Overall Architecture of REMO
Figure 1: Overall Architecture of REMO

Repository and Installation

The tool requires Python 3.8 and Carla version 0.9.15. The source code of REMO, including detailed installation instructions and example scripts, is hosted on GitHub: <https://github.com/RSE-Sheffield/REMO>.

Acknowledgements

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2025-02218761, 50%) and by the Engineering and Physical Sciences Research Council (EPSRC) [EP/Y014219/1].

References

- Arcaini, P., Zhang, X.-Y., & Ishikawa, F. (2022). Less is more: Simplification of test scenarios for autonomous driving system testing. *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 279–290. <https://doi.org/10.1109/ICST53961.2022.00037>
- Chance, G., Ghobrial, A., McAreavey, K., Lemaignan, S., Pipe, T., & Eder, K. (2022). On determinism of game engines used for simulation-based autonomous vehicle verification. *IEEE Transactions on Intelligent Transportation Systems*, 23(11), 20538–20552. <https://doi.org/10.1109/TITS.2022.3177887>
- Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). CARLA: An open urban driving simulator. In S. Levine, V. Vanhoucke, & K. Goldberg (Eds.), *Proceedings of the 1st annual conference on robot learning* (Vol. 78, pp. 1–16). PMLR. <https://proceedings.mlr.press/v78/dosovitskiy17a.html>
- Osikowicz, O., McMin, P., & Shin, D. (2025). Empirically evaluating flaky tests for autonomous driving systems in simulated environments. *2025 IEEE/ACM International Flaky Tests Workshop (FTW)*, 13–20. <https://doi.org/10.1109/FTW66604.2025.00009>
- Shin, D., & Pennada, S. (2024). Towards simplification of failure scenarios for machine learning-enabled autonomous systems. *2024 IEEE 24th International Conference on Software Quality, Reliability, and Security Companion (QRS-c)*, 1089–1090. <https://doi.org/10.1109/QRS-C63300.2024.00143>